

도구의 표준화, MCP

도구의 표준화, MCP

1. MCP란 무엇인가?
2. FastMCP로 간단한 서버 만들기
3. MCP 서버 연결 및 에이전트 활용
4. 외부 MCP 서버 연결하기
5. 라마인덱스(LlamaIndex) MCP
6. 날씨 에이전트 구현

1. MCP란 무엇인가?

- MCP의 등장 배경

- AI 에이전트의 확장성을 제한하는 기존 도구 연결 방식의 한계
- 이를 해결하기 위한 표준 프로토콜인 MCP의 개념을 다룸
- 복잡한 도구 연결 과정을 획기적으로 단순화해 주는 MCP(Model Context Protocol)의 기술
- 이를 통해 에이전트의 확장성을 극대화하는 방법을 알아봄
- 다양한 외부 시스템을 표준화된 방식으로 연결하는 MCP의 역할과 이를 구성하는 핵심 요소인 호스트, 서버, 클라이언트의 아키텍처를 살펴봄

1. MCP란 무엇인가?

● MCP의 기본 개념

- 지금까지 우리는 검색 도구(RAG)나 계산 도구 등을 에이전트에 직접 코드로 구현해서 넣어 줌
- 도구가 1~2개일 때는 이 방식도 괜찮음. 하지만 이 방식은 만들기는 쉽지만, ‘확장성(scalability)’ 벽에 부딪힘
- 만약 여러분이 만든 이 훌륭한 ‘검색 도구’를 동료가 만든 다른 AI 앱에서도 똑같이 쓰고 싶다면, 기존 방식으로는 코드를 복사해서 붙여 넣거나, 각 앱의 사정에 맞춰 코드를 처음부터 다시 짜야 함
- 또한, 구글 드라이브, 슬랙, 노션, 깃허브 등 서비스마다 API 사용법이 다르고, 데이터를 주고 받는 형식도 제각각 임
- 새로운 도구를 추가할 때마다 에이전트의 코드를 수정해야 하는 비효율이 발생
- MCP는 바로 이런 문제를 해결하기 위해 등장한 **개방형 표준 프로토콜**
- 어떤 외부 데이터나 도구(server)든 MCP라는 표준 규격에만 맞춰 개발해 두면, 클로드나 우리가 만들 에이전트(client)가 별도의 복잡한 연동 코드 없이 즉시 가져다 쓸 수 있게 됨

1. MCP란 무엇인가?

- MCP의 기본 개념

- MCP 아키텍처의 세 가지 구성 요소

1. 호스트(host): AI 모델이 탑재된 애플리케이션. 클로드 데스크탑 앱이나, IDE인 커서, 그리고 이번 장에서 우리가 파이썬으로 만들 에이전트가 바로 호스트
2. 서버(server): 에이전트에게 제공할 컨텍스트(context)를 가지고 있는 주체. 여기서 context란 데이터(파일, DB)나 도구(기능)를 의미. “내 컴퓨터의 파일을 읽어줘”, “DB에서 검색해 줘” 같은 기능을 수행하는 독립적인 프로그램. 우리는 FastMCP를 사용해 이 서버를 간편하게 만들 것임
3. 클라이언트(client): 호스트와 서버 사이를 연결하는 다리 역할. 호스트의 요청을 서버로 전달하고, 서버의 응답을 다시 호스트에게 가져다 줌. 랭체인에서는 langchain-mcp-adapters 라이브러리가 이 역할을 수행

- MCP 표준은 호스트, 서버, 클라이언트 세 가지 요소로 정의되지만, 랭체인 에이전트에서의 구현은 다름
- 랭체인 에이전트는 MCP 클라이언트 역할을 하기 때문에, 호스트와 클라이언트가 통합된 형태
- 즉, 에이전트가 LLM과 통신하면서 동시에 MCP 서버에 접속하는 방식

1. MCP란 무엇인가?

- MCP의 기본 개념
 - 기존 우리가 랭체인으로 만들던 에이전트와는 무엇이 달라지는가?

항목	기존 방식(랭체인 도구)	FastMCP 방식
도구 위치	에이전트 코드 내부	독립적인 서버
설정	복잡한 통합 코드	mcp.tool 데코레이터
확장	에이전트 코드 수정 필요	새 서버 추가만으로 확장
재사용	랭체인에만 적용	모든 MCP 클라이언트에 적용
라이브러리	랭체인	langchain-mcp-adapters + mcp

1. MCP란 무엇인가?

● 왜 MCP가 필요한가?

- “그냥 기존처럼 랭체인 도구로 만들면 되지 않은가?”라고 물으실 수 있고, 맞습니다. 혼자 개발하는 작은 프로젝트라면 기존 방식이 더 빠를 수도 있음
- 하지만, 프로젝트 규모가 커지거나 협업이 필요한 상황에서는 이야기가 달라짐
- 가상의 팀 프로젝트 상황을 생각해 봄
 - 팀원 A: “나는 파일 읽고 쓰는 기능을 만들께”
 - 팀원 B: “나는 사내 DB에서 고객 정보 조회하는 기능을 만들께”
 - 팀원 C: “나는 이 두 기능을 합쳐서 에이전트를 만들께”
- MCP 없이 개발한다면, 팀원 C는 팀원 A와 팀원 B가 작성한 코드를 모두 에이전트 파일 하나에 통합해야 함
- 코드가 길어지고 복잡해지는 것은 물론, A가 코드를 수정하면 C도 전체 에이전트를 다시 배포해야 하는 의존성 문제가 발생
- 하지만 FastMCP를 도입하면 상황이 개선됨

1. MCP란 무엇인가?

- 왜 MCP가 필요한가?

- 개선 상황

- 팀원 A는 파일 읽고 쓰는 기능을 `mcp.tool` 데코레이터로 정의하고 FastMCP 서버로 실행함(`file_server.py`)
 - 팀원 B는 DB 조회 기능을 `mcp.tool` 데코레이터로 정의하고 FastMCP 서버로 실행함(`db_server.py`)
 - 팀원 C의 에이전트는 `MultiServerMCPClient`로 이 서버들을 ‘연결’만 하면 됨

- 각자가 만든 도구는 서로의 코드에 영향을 주지 않으며, FastMCP 서버를 처음에 잘 만들어 두면 이후 다른 프로젝트에서도 쉽게 재사용할 수 있음
 - 심지어 랭체인이 아닌 다른 AI 앱에서도 같은 MCP 서버를 사용할 수 있음
 - 즉, 이번 장은 그동안 만든 에이전트를 ‘확장 가능한 시스템’으로 전환하는 단계
 - 에이전트가 외부 MCP 서버와 표준화된 방식으로 연결되어, 기존의 복잡한 통합 코드 없이도 다양한 도구와 데이터를 활용할 수 있게 됨

2. FastMCP로 간단한 서버 만들기

- FastMCP는 랭체인 생태계에서 MCP 서버를 가장 빠르고 직관적으로 만들 수 있게 도와주는 도구
- 복잡한 프로토콜 규격을 몰라도, 파이썬 함수 위에 데코레이터만 붙이면 즉시 MCP 서버로 변환해 줌

● 파일 관리 서버 만들기

- 파일을 읽고, 쓸 수 있는 기능
 - 에이전트가 모델이 답변한 내용을 파일로 생성해 주도록 하는 역할
- 환경 설정

```
# 가상 환경 생성
$ python -m venv .venv

# 윈도우 가상 환경 활성화
$ source .venv/Scripts/activate # 맥 환경에서는 Scripts를 bin으로 변경
(.venv)
$ python -m pip install --upgrade pip

# 패키지 설치
(.venv)
$ pip install langchain langchain-openai mcp langchain-mcp-adapters
```

2. FastMCP로 간단한 서버 만들기

- 환경 설정
 - mcp 패키지는 엔트로픽에서 제공하는 공식 MCP Python SDK
 - langchain-mcp-adapters 패키지는 랭체인 에이전트가 MCP 서버와 쉽게 통신할 수 있도록 도와주는 연결 도구
 - 첫 번째로 만들 서버는 '파일 관리자'
 - 에이전트가 내 컴퓨터의 파일을 읽고(read_file), 새로운 내용을 저장(write_file)할 수 있도록 도와주는 도구 서버
 - 프로젝트 폴더에 'file_server.py' 파일을 생성
- 서버 객체 생성
 - 가장 먼저 할 일은 FastMCP 객체를 생성하는 것
 - 이 객체는 도구들을 담을 그릇이자, 서버의 이름표 역할
 - 여기서 정의한 이름(FileManager)은 나중에 클라이언트가 서버를 식별할 때 사용
 - 마치 에이전트의 도구 이름을 정의하는 것과 같은 역할

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("FileManager")
```

2. FastMCP로 간단한 서버 만들기

- ‘파일 읽기’ 도구 만들기

```
@mcp.tool('read_file', description="파일을 읽어 내용을 반환합니다.")
def read_file(path):
    """
    지정된 경로의 파일을 읽어 내용을 반환합니다.

    Args:
        path: 읽을 파일의 경로 (예: './data/report.txt')
    """
    try:
        with open(path, 'r', encoding='utf-8') as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: '{path}' 파일을 찾을 수 없습니다."
    except Exception as e:
        return f"Error: 파일을 읽는 중 오류가 발생했습니다. {str(e)}"
```

- 에이전트가 사용할 첫 번째 도구, read_file 함수를 만들
- 함수 위에 mcp.tool 데코레이터를 붙임
- 이것만 붙이면 일반 파이썬 함수가 ‘MCP 도구’로 변신
- 이때 사용할 수 있는 인자는 랭체인의 tool 데코레이터와 유사한 역할을 수행
- 항상 description과 docString의 역할을 헷갈리지 않도록 유의

2. FastMCP로 간단한 서버 만들기

- ‘파일 쓰기’ 도구 만들기

```
@mcp.tool('write_file', description="지정된 경로에 내용을 파일로 저장합니다.")
def write_file(path, content):
    """
    지정된 경로에 내용을 파일로 저장합니다.
    (이미 존재하는 파일이면 덮어씁니다.)

    Args:
        path: 저장할 파일의 경로
        content: 파일에 작성할 텍스트 내용
    """
    try:
        with open(path, 'w', encoding='utf-8') as f:
            f.write(content)
        return f"Successfully wrote to '{path}'"
    except Exception as e:
        return f"Error: 파일 저장 중 오류가 발생했습니다. {str(e)}"
```

- 파일에 내용을 저장하는 write_file 도구 만들기
- 마찬가지로 mcp.tool 데코레이터를 붙여줌
- 이 도구의 역할은 순수하게 ‘지정된 위치(path)에 넘겨받은 내용(content)을 작성’ 하는 것

2. FastMCP로 간단한 서버 만들기

- 서버 실행 진입점 만들기

```
if __name__ == "__main__":  
    mcp.run(transport="stdio")
```

- 마지막으로 이 파일이 직접 실행될 때 서버가 켜지도록 설정
- 여기서 mcp.run(transport="stdio")가 핵심
- 이는 서버를 인터넷(HTTP)이 아닌, 표준 입출력(Standard I/O) 방식으로 실행하겠다는 뜻
- 로컬에서 에이전트와 빠르고 안전하게 통신하기 위해 주로 사용되는 방식
- 코드 작성이 끝났다면, 터미널에서 서버를 실행

```
(.venv)  
$ python file_server.py
```

- 오류 없이 정상적으로 실행된다면 아무런 반응 없이 커서가 멈춰 있는 것처럼 보일 것임
- 서버가 정상적으로 실행되어 '누군가 말을 걸어주길 기다리고 있는 상태(listening)'
- stdio 방식은 화면에 로그를 출력하는 대신, 입력과 출력을 통신 채널로 사용하기 때문에 눈에는 아무것도 보이지 않는 것이 정상
- 이제 도구를 제공하는 서버가 준비되었음
- 이렇게 생성한 MCP 서버는 다음 절에서 배울 MultiServerMCPClient가 대신 실행해 줄 것이기 때문에, 서버를 종료(ctrl + c)해 줌

3. MCP 서버 연결 및 에이전트 활용

- 도구(서버)가 준비되었으니, 다음은 이 도구를 사용할 에이전트를 만드는 것
- 이전 장까지는 에이전트 코드 안에 도구 정의가 섞여 있었지만, 이제는 다름
- 에이전트는 도구가 어떻게 만들어졌는지 알 필요가 없음
- 그저 “FileManager 서버에 연결해 줘” 라고 요청만 하면, 그 안에 있는 기능(read_file, write_file)을 자동으로 가져와 사용할 수 있게 됨

● MCP 서버의 도구를 에이전트에 불러오기

- 새로운 파이썬 파일 ‘agent_with_mcp.py’를 생성하고, 에이전트가 서버에 접속하여 도구를 불러오는 과정을 구현해 보겠음
 - 여기서 핵심은 MultiServerMCPClient
 - 이 클라이언트는 하나가 아니라 여러 개의 MCP 서버를 동시에 연결하여 관리할 수 있는 강력한 기능 제공
-
- 오픈AI API 키 등록
 - 에이전트의 모델을 위해 오픈AI API 키를 ‘.env’ 파일에 작성

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- 라이브러리 import 및 설정

```
import asyncio
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.messages import AIMessage, ToolMessage
from langchain_mcp_adapters.client import MultiServerMCPClient
from dotenv import load_dotenv

load_dotenv()

model = ChatOpenAI(model="gpt-5-nano", temperature=0)
```

- MCP 통신은 기본적으로 비동기(asynchronous) 방식으로 작동함
- 따라서 asyncio 라이브러리를 사용해야 함
- asyncio는 async/await 구문을 사용하여 동시성 코드를 작성하는 라이브러리

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기
 - MCP 클라이언트 생성 및 서버 연결

```
async def main():  
    """스트리밍 모드로 MCP 에이전트 실행"""  
    # 1. MCP 클라이언트 생성: 서버 정보 등록  
    client = MultiServerMCPClient({  
        "file_manager": {  
            "transport": "stdio",  
            "command": "python",  
            "args": ["file_server.py"]  
        }  
    })  
  
    # 2. 서버와 연결하고 도구 가져오기  
    tools = await client.get_tools()
```

- 이제 클라이언트를 생성하고 우리가 만든 'file_server.py' 를 등록함
- 이를 위해 main 함수를 작성할 때 async 구문을 함수 이름 앞에 작성함. 이 키워드 만으로 main 함수는 논블로킹 형식으로 동작할 수 있게 됨
- 함수 내부에 MultiServerMCPClient 클래스로 클라이언트를 생성

continue ...

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- MCP 클라이언트 생성 및 서버 연결

- 설정값 중 `command`와 `args`는 터미널에서 서버를 실행할 때 쓰는 명령어와 동일. 즉, 에이전트가 실행될 때 내부적으로 `'python file_server.py'` 명령어를 대신 실행해서 연결 통로를 뚫어주는 원리
- 이전에 생성한 MCP 서버는 특별한 추가 인자 없이 파이썬의 기본 실행 명령어 `'python file_server.py'` 방식으로 실행. 만약, 파일 경로가 다르다면 그에 맞게 경로도 수정
- 이번 실습에서는 1개의 MCP 서버만 사용하지만, 만약 다양한 MCP 서버를 가지고 있거나 활용할 수 있다면 이곳에 얼마든지 추가할 수 있음
- 서버에는 다양한 도구들(`read_file`, `write_file`)이 포함되어 있음. 그리고 이 도구들을 에이전트에게 전달하기 위해 불러오는 과정이 필요
- 여러 개의 서버에 연결하여 각 도구들을 불러오는 데는 분명 시간이 필요. 이를 위해 `async/await` 구문을 사용함
- `client.get_tools` 메서드는 생성한 클라이언트가 가진 모든 도구를 불러오는 함수
- 이때, `await` 구문을 작성하여 작업이 진행되는 동안 다른 업무가 진행될 수 있도록 처리함
- 마지막으로 모든 도구를 불러왔다면 에이전트에 전달하는 것으로 MCP와 에이전트의 연결은 끝

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- 에이전트 생성 및 실행을 위한 준비

```
async def main():  
  
    ... (생략) ...  
  
    agent = create_agent(  
        model=model,  
        tools=tools,  
        system_prompt="""  
        당신은 유능한 파일 관리자입니다.  
        """  
    )
```

- 도구를 가져왔으니, 이제 이 도구를 사용할 에이전트를 생성하고 실제 업무를 시켜 봄
- 테스트를 위해 프로젝트 폴더에 간단한 내용을 담은 “sample.txt” 파일을 미리 만들어 둬(혹은, 본인이 요약하고 싶은 내용을 sample.txt 파일에 직접 작성해도 됨)
- 에이전트에게 이 파일을 읽고, 내용을 요약해서 새로운 파일로 저장하라는 복합적인 임무를 부여할 것임
- 파일의 위치는 편의를 위해 agent_with_mcp.py 파일과 같은 위치에 둬
- 우선 에이전트에 모델, 도구, 시스템 프롬프트를 전달함
- 도구들은 get_tools 메서드를 통해 받아오는 정보를 그대로 전달해 에이전트 생성을 마칩

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기
 - 에이전트 호출

```
async def main():  
    ... (생략) ...  
  
    query = "sample.txt를 읽고 summary.md에 요약을 저장해줄래?"  
  
    print(f"사용자: {query}\n")  
  
    # astream() 사용 (비동기 스트리밍)  
    async for chunk in agent.astream(  
        {"messages": [{"role": "user", "content": query}]},  
        stream_mode="values"  
    ):  
        if "messages" not in chunk:  
            continue  
  
        latest_msg = chunk["messages"][-1]  
  
        if latest_msg.__class__.__name__ == "AIMessage":  
            # 도구 호출이 있으면 = 사고 과정 (Reasoning)  
            if latest_msg.response_metadata.get('finish_reason') == 'tool_calls':  
                # 도구 호출 중  
                if latest_msg.tool_calls:  
                    for tc in latest_msg.tool_calls:  
                        print(f"[Reasoning] 도구 호출: {tc['name']}")  
                        print(f"    [Acting] {tc['name']} {tc['args']}...\n")  
            # 도구 호출이 없고 content가 있으면 = 최종 답변 (Finish)  
            elif latest_msg.content:  
                print(f"[Finish] {latest_msg.content[:150]}...\n")  
        elif latest_msg.__class__.__name__ == "ToolMessage":  
            print(f"[Observation] {latest_msg.content[:150]}...\n")
```

```
if __name__ == "__main__":  
    asyncio.run(main())
```

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- 에이전트 호출

- 이제 남은 것은 질문과 에이전트의 동작을 출력해 보는 것
- 이때, stream 메서드 대신 astream 메서드를 사용함
- 또한 반복문 역시도, async 구문을 사용하여 정의
- astream 메서드는 비동기 제너레이터(async generator)를 반환하기 때문
- astream 메서드를 사용해야 하는 이유
 1. LLM을 호출: 네트워크 요청으로 시간이 소요됨
 2. 도구 호출(MCP 서버 통신): MCP 서버와 통신하므로 시간이 소요됨
 3. LLM을 다시 호출(도구의 결과를 토대로 LLM 재호출): 마찬가지로 시간이 소요됨
- 일반적인 상황이라면 코드는 단일 작업만 진행하므로 stream 메서드만으로 충분. 하지만 MCP 클라이언트가 도구를 호출하는 동안 다른 I/O 작업을 동시에 처리해야 하는 경우가 있음
- 특히 MultiServerMCPClient로 여러 MCP 서버를 동시에 관리하거나, 나중에 MCP 서버를 추가할 때 비동기 작업이 필요해 짐
- MCP 서버를 추가할 때마다 더 많은 동시 I/O 작업이 발생하므로, astream 메서드의 필요성이 증가함

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- 출력 결과 확인

사용자: sample.txt를 읽고 summary.md에 요약을 저장해줄래?

[Reasoning] 도구 호출: read_file

[Acting] read_file {'path': 'sample.txt'}...

[Observation] [{'type': 'text', 'text': '마이클 샌델 하버드 대학 교수가 저술한 정의란 무엇인가는 현대 정치 철학의 흐름을 대중적인 언어로 풀어내며 정의에 대한 근본적인 물음을 던지는 책으로 출간 이후 전 세계적으로 큰 반향을 일으켰으며 특히 한국 사회에서 인문학 서적으로는 이례적인 베스트셀러가 되어 정의 열풍을 ... (생략) ...'}]

[Reasoning] 도구 호출: write_file

[Acting] write_file {'path': 'summary.md', 'content': '샌델의 정의란 무엇인가?는 현대 정치철학의 흐름을 대중적으로 설명하고, 정답보다 꾸준한 질문과 토론을 통해 도덕적 판단의 중요성을 강조한다. 책은 정의에 관한 세 가지 큰 흐름을 제시한다: 1) 공리주의—최대 다수의 행복 추구, 그러나 개인의 존엄과 권리 침해 위험과 ... (생략) ...'}]

- 에이전트가 FastMCP 서버의 도구들을 잘 가지고 왔는지 확인
- 에이전트는 먼저 read_file 도구를 호출하여 sample.txt의 내용을 읽는데 성공
- 별도로 서버가 실행 중이 아님에도, MultiServerMCPClient에 작성한 실행 방법에 따라 서버로부터 도구를 가져와 실행한 것임
- 그렇게 내용을 확인한 에이전트는 이어서 write_file 도구도 호출
- 이때, 모델을 통해 요약한 내용을 content 인자에 넘겨주고 있음
- 작성해야 하는 파일의 위치도 사용자가 지정한 이름으로 잘 작성하여 path 인자에 넘겨주고 있음

3. MCP 서버 연결 및 에이전트 활용

- MCP 서버의 도구를 에이전트에 불러오기

- 출력 결과 확인

[Observation] [{'type': 'text', 'text': "Successfully wrote to 'summary.md'", 'id': 'lc_8a2a29b2-4b7b-40e3-b89f-b7345e33e08e'}]...

[Finish] 확인했습니다. sample.txt를 읽어 summary.md에 요약을 저장했고, 현재 summary.md의 내용은 아래와 같습니다.

샌델의 정의란 무엇인가?는 현대 정치철학의 흐름을 대중적으로 설명하고, 정답보다 꾸준한 질문과 토론을 통해 도덕적 판단의 중요성 을 강조...

- 도구 호출을 마치고 난 후, 에이전트는 사용자에게 남길 마지막 메시지를 결정함
 - 어떤 작업을 했으며 본인이 요약한 내용에 대해서 한 번 더 정리한 결과를 알려주고 있음
 - 에이전트는 맡은 업무를 잘 수행하여 'summary.md' 파일을 생성해 줌

4. 외부 MCP 서버 연결하기

- 앞서 우리는 파일을 다루는 서버를 직접 만들었지만, 실무에서 마주할 슬랙, 깃허브, 구글 드라이브 같은 거대 서비스들의 연동 도구를 매번 직접 만드는 것은 불가능에 가까움
- 각 서비스의 복잡한 API 문서를 공부하고, 인증 과정을 구현하는 데만 며칠 밤을 새워야 할 것임
- MCP 생태계에는 전 세계의 개발자들이 미리 만들어 둔 수많은 '표준 서버'들이 존재함
- 파이썬 라이브러리를 설치하듯, 이 서버들을 가져와 설정 파일에 한 줄 추가하는 것만으로 에이전트에게 다양한 기능을 부여할 수 있음
- 복잡한 코딩 없이, 터미널 명령어와 설정 변경만으로 에이전트의 능력을 확장하는 실습 진행

● 외부 서버 설치하기

- 사용해 볼 도구는 현재 시간을 정확하게 다루는 mcp-server-time 임
- mcp-server-time은 시간 및 시간대 변환 기능을 제공하는 MCP 서버
- 이 서버를 통해 LLM은 현재 시간 정보를 가져오고 IANA 시간대 이름을 사용하여 시간대를 변환할 수 있으며, 시스템 시간대 자동 감지 기능을 사용할 수 있음
- 이 서버는 pip를 통해 간단하게 설치하고 사용할 수 있음

- <https://pypi.org/project/mcp-server-time>

mcp-server-time 2026.1.26

`pip install mcp-server-time` 

4. 외부 MCP 서버 연결하기

● 외부 서버 설치하기

- 먼저 pip install 명령어로 mcp-server-time 도구 설치 및 실행

```
# 설치 및 실행
```

```
(.venv)
```

```
$ pip install mcp-server-time
```

```
$ python -m mcp_server_time
```

- 실행 명령어 작성시 주의 : 도구 이름과 실행이 다름
- 정상적으로 설치를 마친 후, 명령어를 입력하면 아무 일도 벌어지지 않고 멈춘 것처럼 보임
- 앞선 2절 ‘파일 관리 서버 만들기’ 에서 보았듯이 다음 요청을 대기하고 있는 상태
- 확인을 완료했으면 서버 종료(ctl + c)

4. 외부 MCP 서버 연결하기

- 외부 서버 사용하기

- MCP 서버 추가

```
async def main():
    """스트리밍 모드로 MCP 에이전트 실행"""
    # 1. MCP 클라이언트 생성: 서버 정보 등록
    client = MultiServerMCPClient({
        "file_manager": {
            "transport": "stdio",
            "command": "python",
            "args": ["file_server.py"]
        },
        "time_server": {
            "transport": "stdio",
            "command": "python",
            "args": ["-m", "mcp_server_time", "--local-timezone=Asia/Seoul"]
        }
    })
```

... (생략) ...

- 기존에 작성했던 'agent_with_mcp.py' 파일을 열어 MultiServerMCPClient 설정 부분만 수정하면 됨
- 이전에 만든 file_manager 서버는 그대로 유지한 채, 그 아래에 time_server라는 이름으로 새로운 연결 정보를 추가
- 이때, 서버를 실행시킬 때 필요한 인자 정보를 "args": ["-m", "mcp_server_time", "--local-timezone=Asia/Seoul"]으로 작성하여 서울 표준시 기준으로 동작하도록 작성

4. 외부 MCP 서버 연결하기

- 외부 서버 사용하기

- 시스템 프롬프트 수정

```
async def main():  
  
    ... (생략) ...  
  
    agent = create_agent(  
        model=model,  
        tools=tools,  
        system_prompt="""  
        당신은 유능한 파일 관리자입니다.  
        사용자가 파일 생성에 대해 요청하면, 파일을 생성하고,  
        파일 요약을 요청하면, 파일을 읽고 요약을 작성합니다.  
        그리고 반드시 파일 생성시에는 작성 일자를 포함해야 합니다.  
        """  
    )  
    ... (생략) ...
```

- 에이전트에게 도구를 추가해 주었으니, 어떻게 사용할지 알려주어야 함

4. 외부 MCP 서버 연결하기

- 외부 서버 사용하기

- 출력 결과 확인

사용자: sample.txt를 읽고 summary.md에 요약을 저장해줄래?

[Reasoning] 도구 호출: read_file

[Acting] read_file {'path': 'sample.txt'}...

[Reasoning] 도구 호출: get_current_time

[Acting] get_current_time {'timezone': 'Asia/Seoul'}...

[Observation] [{'type': 'text', 'text': '{\n "timezone": "Asia/Seoul",\n "datetime": "2026-04-11T23:20:51+09:00",\n "day_of_week": "Saturday",\n "is_dst": false\n}', 'id': 'lc_a8741594-f32f-4e17-8b0e-1591fe979fbf'}]...

- 에이전트는 이전과 동일하게 먼저 사용자의 질문을 받고, 파일을 찾아 읽기 위해 read_file 도구 호출
- 에이전트는 이어서 get_current_time 도구를 호출. 현재 시간을 알아내기 위함
- mcp-server-time이 정상적으로 동작 중이라는 뜻

4. 외부 MCP 서버 연결하기

● 외부 서버 사용하기

• 출력 결과 확인

[Reasoning] 도구 호출: write_file

[Acting] write_file {'path': 'summary.md', 'content': '작성일: 2026-05-13 23:35:07 (Asia/Seoul)\\n\\n요약\\n- 이 책은 정의에 관한 근본적 물음을 제시하며 소크라테스식 대화법으로 독자의 사고를 자극합니다.\\n- 정의의 세 가지 주요 관점: \\n 1) 공리주의: 최대다수의 행복. 단점으로 ... (생략) ...

[Observation] [{'type': 'text', 'text': "Successfully wrote to 'summary.md'", 'id': 'lc_93ee0fa6-897d-4d05-abcc-4920468dec72'}]...

[Finish] 요약이 작성되었습니다. 파일 경로: summary.md
작성일: 2026-05-13 23:35:07 (Asia/Seoul)

summary.md의 내용은 다음과 같습니다:

- 이 책은 정의에 관한 근본적 물음을 제시하며 소크라테스식 대화법으로 독자의 사고를 자극합니다....

- 날짜 확인을 마친 에이전트는 모델이 생성한 요약문을 summary.md 파일에 작성하기 위해 write_file 도구 호출
- 도구 호출 결과 정상적으로 파일이 생성된 것을 확인하고는 이제 모든 업무를 마쳤다고 판단했는지 더 이상의 도구 호출 없이 마지막 답변을 생성하고 업무를 마칩
- 마지막 답변은 요약 파일을 어떻게 작성했는지 설명
- summary.md 파일에서 작성일 날짜가 있는지 확인

4. 외부 MCP 서버 연결하기

- 이번 chapter의 출력 결과를 확인하면서

- 에이전트에게 “A 도구를 쓰고 B 도구를 써” 라고 코딩하지 않았음
- 단지 표준 규격(MCP)에 맞춰진 도구들을 연결시켜 주었을 뿐
- 에이전트는 “파일 생성시에는 작성 일자를 포함해야 합니다”라는 지시 사항을 해결하기 위해 외부에서 가져온 mcp-server-time 을 스스로 선택했고, “저장해 줘”라는 요청을 해결하기 위해 우리가 만든 file_serve를 연계해서 사용함
- 이처럼 MCP를 활용하면, 자신이 만든 기능과 남이 만든 기능을 레고 블록처럼 손쉽게 조립하여 에이전트를 구축할 수 있음

5. 라마인덱스(LlamaIndex) MCP

- 모든 외부 도구와의 통신을 동일한 프로토콜로 표준화해서 도구의 이름, 설명, 입력 매개변수, 반환 형식 등을 일관된 방식으로 정의
- 이는 JSON-RPC 2.0을 기반으로 한 구조화된 메시지 교환 방식으로 구현
 - MCP는 툴을 단순한 데이터 자원이 아니라 실행 가능한 기능으로 바라봄
 - 즉, 외부 도구와의 통신은 단순히 정보를 조회하는 것을 넘어서 특정 동작을 함수처럼 호출하고 그 결과를 받아야 하는 구조를 필요로 함
 - JSON-RPC는 명령 자체를 중심으로 메시지를 구성할 수 있어 표현이 훨씬 더 명확하고 직관적임
 - 매개변수 전달과 결과 반환이 구조화되어 있어 복잡한 함수 호출을 체계적으로 처리할 수 있음
- 통합 과정 또한 간소화
 - 새로운 도구를 추가할 때는 MCP 서버만 구현하면 되며, 클라이언트인 AI 어플리케이션은 MCP 프로토콜 하나만 이해하면 모든 도구를 사용할 수 있게 됨

5. 라마인덱스(LlamaIndex) MCP

- 라마인덱스(LlamaIndex)

- LLM 기반 어플리케이션을 개발할 때 활용할 수 있는 데이터 프레임워크
- MCP 기반 에이전트 구성, 문서 인덱싱, 검색 및 도구 연동과 같은 핵심 기능을 비교적 간단하게 실습할 수 있음
- 라마인덱스는 랭체인과 함께 LLM 어플리케이션 개발 분야의 주요 프레임워크로 자리매김하고 있음
- 설치

```
# 패키지 설치  
(.venv)  
$ pip install llama-index llama-index-tools-mcp
```

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

1. 문서 검색 기능을 MCP 툴(Tool)로 등록
2. 기존 인덱스 기반 검색기(query engine)를 MCP에 연결
3. 최종적으로 MCP 서버를 실행해 모델이 사용할 수 있는 서비스로 제공

• 어댑터를 활용한 툴 등록

- 모델이 스스로 문서를 탐색하고, 필요한 정보를 외부에서 가져올 수 있도록 모델이 직접 호출할 수 있는 외부 툴 등록
- 어댑터는 파이썬 함수와 MCP 툴 사이를 연결하는 중간 처리 계층으로, 일반적인 함수를 LLM이 호출할 수 있는 형식의 MCP도구로 변환하는 역할
- 함수의 입력/출력 포맷을 JSON-RPC 기반의 MCP 툴 형식으로 변환하고, 툴에 필요한 메타 정보(이름, 설명, 파라미터 등)를 추출해서 LLM이 이해할 수 있도록 명세화
- MCP에서는 이 어댑터 기능을 앞서 살펴본 `@mcp.tool()` 데코레이터로 매우 간단하게 제공
- `server.py`의 간단한 텍스트 요약 예시 기능을 MCP 도구로 정의

5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- server.py

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("MyServer")

@mcp.tool()
def summarize_text(text: str) -> str:
    """긴 텍스트를 요약해 간결하게 반환합니다."""
    return text[:100] + "..."

if __name__ == "__main__":
    mcp.run(transport="sse")
```

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- MCP 서버에 툴이 정상적으로 등록됐는지 여부는 별도의 스크립트로 점검할 수 있음
- list_tools.py 파일을 만들어 현재 MCP 서버에 어떤 도구들이 등록돼 있는지 출력

```
import json
from llama_index.tools.mcp import BasicMCPClient, McpToolSpec

# MCP 서버 연결
mcp_client = BasicMCPClient("http://localhost:8000/sse")
mcp_tool_spec = McpToolSpec(client=mcp_client)

tools = mcp_tool_spec.to_tool_list()
for tool in tools:
    tool_dict = {
        "name": tool.metadata.name,
        "description": tool.metadata.description,
        "parameters": json.loads(tool.metadata.fn_schema_str)
    }
    print(json.dumps(tool_dict, indent=2, ensure_ascii=False))
```

- BasicMCPClient 객체를 활용해 이후 server.py를 기반으로 포트 번호 8000에서 실행할 MCP 서버에 연결
- McpToolSpec을 사용해 서버에 등록된 툴 목록을 가져오고, 각 툴의 이름(name), 설명(description), 파라미터 정보(parameters)를 JSON 형식으로 출력하도록 구성

5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- MCP 서버 실행

```
(.venv)  
$ python server.py
```

- 실행 결과

```
INFO: Started server process [26152]  
INFO: Waiting for application startup.  
INFO: Application startup complete.  
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- list_tools.py를 실행해 등록된 툴 목록을 출력

```
(.venv)
```

```
$ python list_tools.py
```

- 실행 결과

```
{
  "name": "summarize_text",
  "description": "긴 텍스트를 요약해 간결하게 반환합니다.",
  "parameters": {
    "properties": {
      "text": {
        "description": "",
        "title": "Text",
        "type": "string"
      }
    },
    "required": [
      "text"
    ],
    "type": "object"
  }
}
```

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- MCP 인스펙터(inspector)
 - 앞서 MCP 서버를 구현하고, 별도의 list_tools.py 스크립트로 등록된 툴 목록을 확인하는 방법을 살펴봄
 - 이러한 작업에 MCP 인스펙터 도구를 활용하면 더 직관적이고 편리하게 확인할 수 있음
 - 인스펙터 공식 깃허브 저장소: <https://github.com/modelcontextprotocol/inspector>
 - 웹 기반 UI를 제공하는 개발 도구로, MCP 서버를 테스트하고 디버깅하기 위한 도구
 - MCP 인스펙터를 이용하여 다음과 같은 작업 수행 가능
 - MCP 서버에 현재 등록된 툴 목록을 시각적으로 탐색
 - 각 툴의 이름, 설명, 파라미터 스키마 확인
 - 서버와의 연결 상태 점검 및 디버깅 로그 확인
 - MCP 인스펙터는 npm으로 간단하게 설치할 수 있음 - npm 설치가 선행되어야 함(다음 페이지)

```
$ npm install -g @modelcontextprotocol/inspector
```

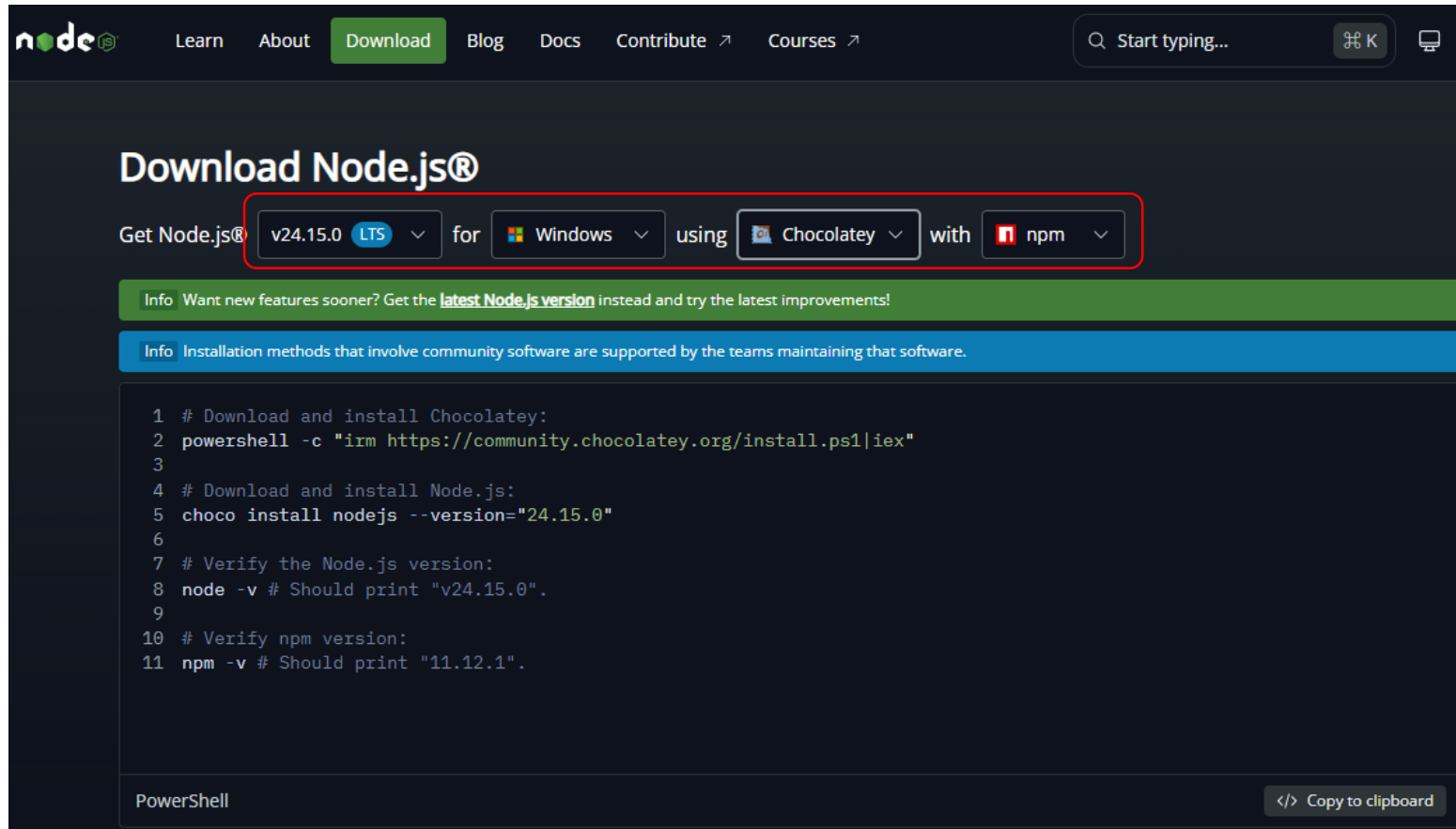
5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- MCP 인스펙터(inspector)

- npm 설치 방법

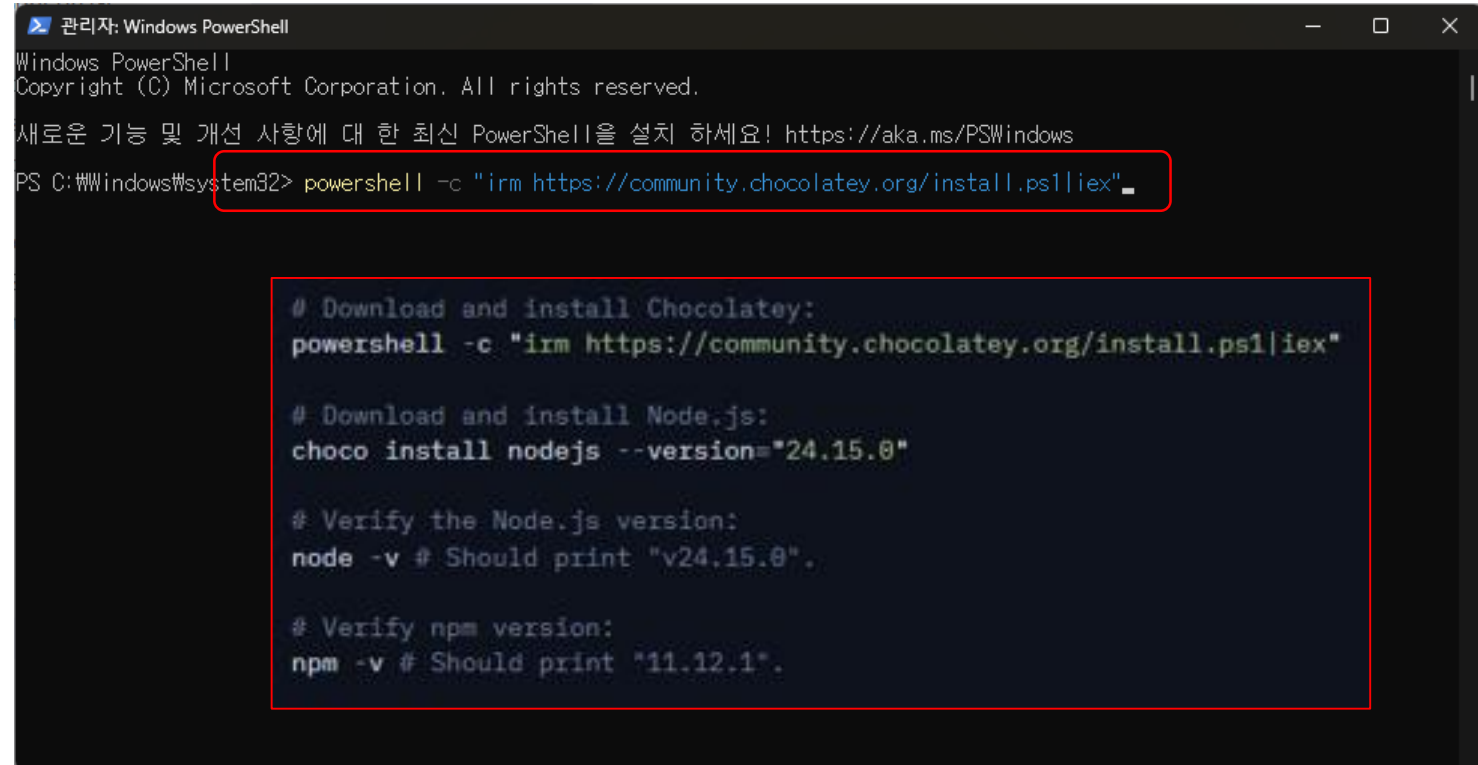
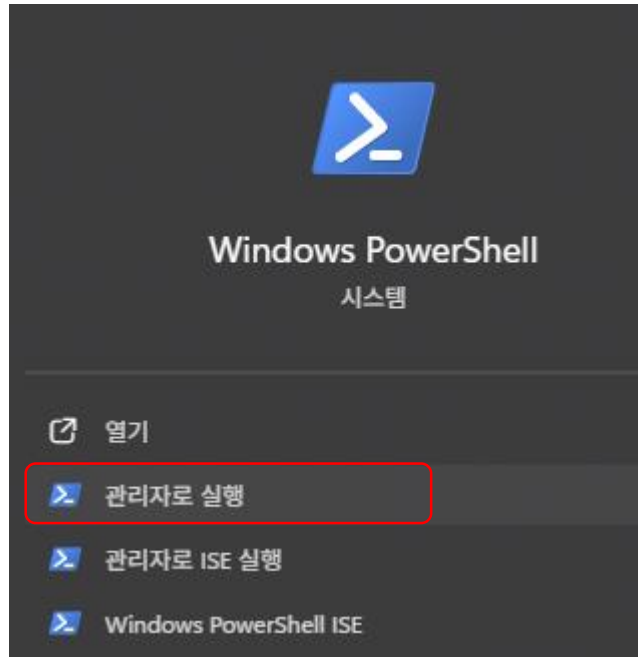
1. Node.js 공식 사이트(<https://nodejs.org>)에 접속해 다운로드 메뉴 클릭.



5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- MCP 인스펙터(inspector)
 - npm 설치 방법
- 2. 설치 옵션에서 LTS, Windows, Chocolatey, npm 선택
- 3. 안내된 설치 스크립트를 복사해서 파워 셸에서 실행



5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

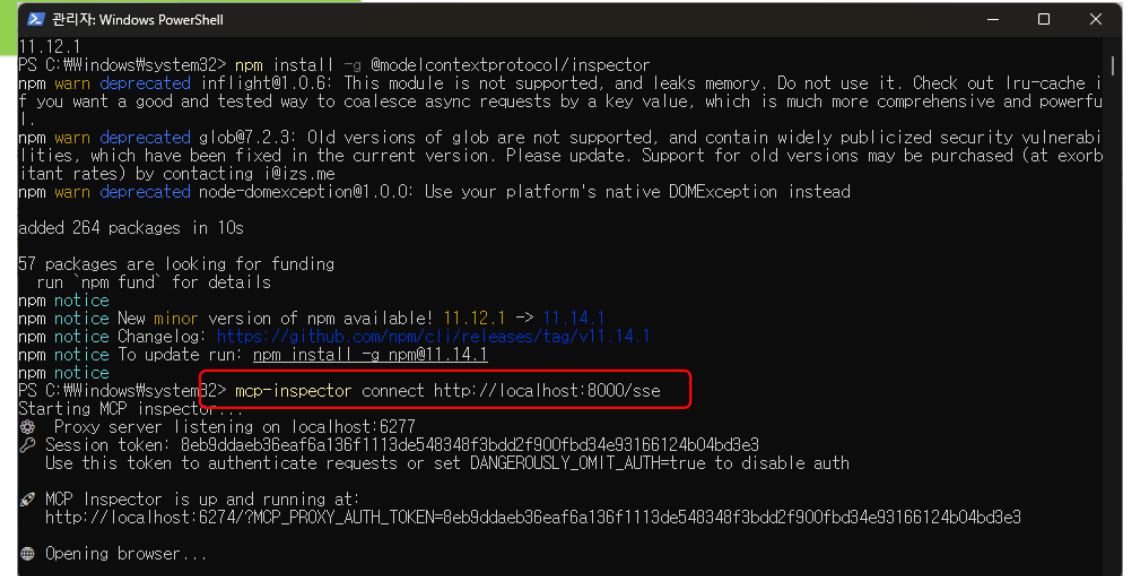
- MCP 인스펙터(inspector)
 - 이제 MCP 서버를 실행하고, 인스펙터가 해당 서버에 접근할 수 있도록 설정

(.venv)

```
$ python server.py
```

- MCP 서버가 정상적으로 실행되면 MCP 인스펙터를 실행해 해당 서버에 연결

```
$ mcp-inspector connect http://localhost:8000/sse
```

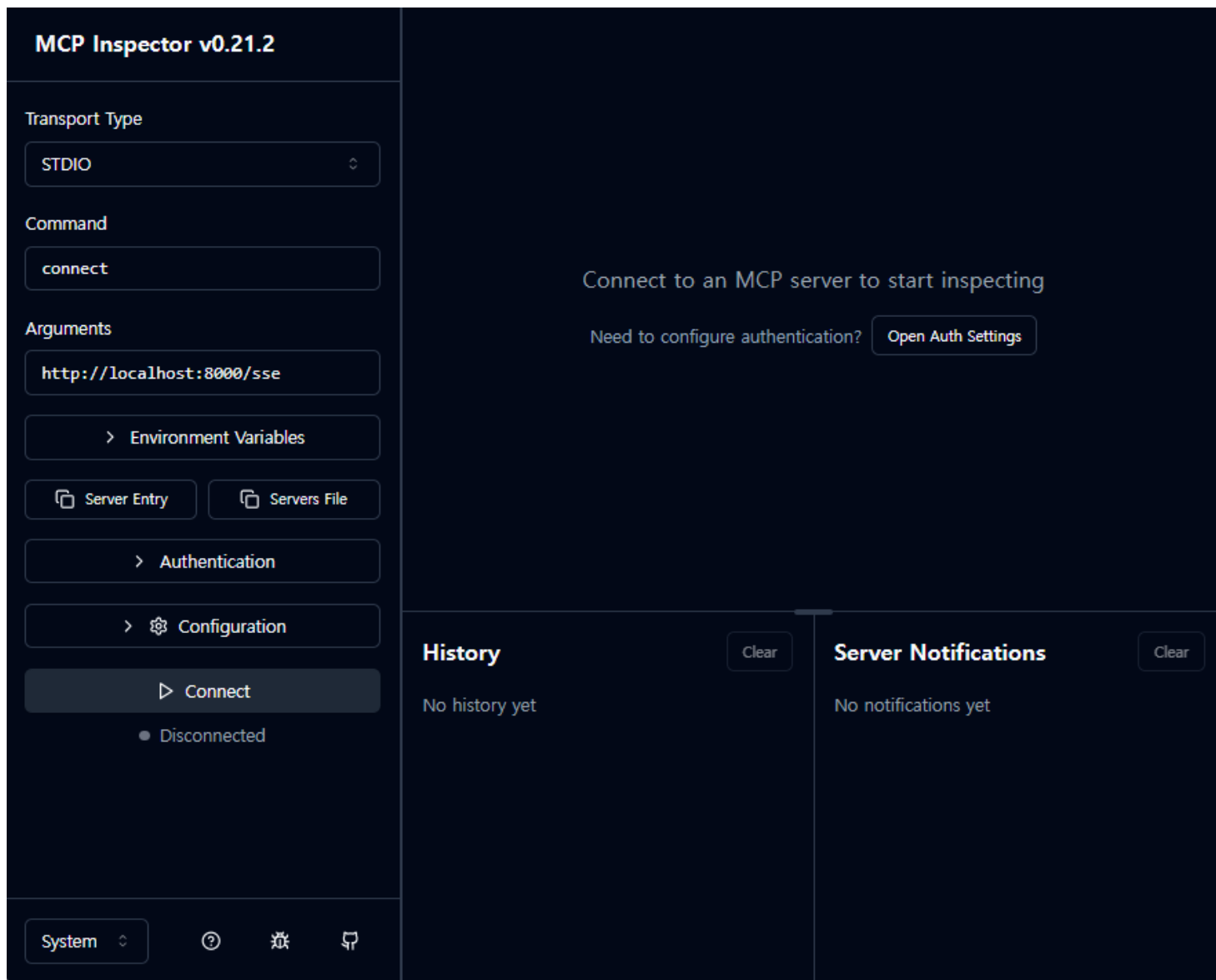


```
관리자: Windows PowerShell
11.12.1
PS C:\Windows\system32> npm install -g @modelcontextprotocol/inspector
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache i
f you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
l.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabi
lities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorb
itant rates) by contacting i@izs.me
npm warn deprecated node-domexception@1.0.0: Use your platform's native DOMException instead
added 264 packages in 10s
57 packages are looking for funding
  run `npm fund` for details
npm notice
npm notice New minor version of npm available! 11.12.1 -> 11.14.1
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.14.1
npm notice To update run: npm install -g npm@11.14.1
npm notice
PS C:\Windows\system32> mcp-inspector connect http://localhost:8000/sse
Starting MCP inspector...
Proxy server listening on localhost:6277
Session token: 8eb9ddaeb36eaf6a136f1113de548348f3bdd2f900fbd34e93166124b04bd3e3
Use this token to authenticate requests or set DANGEROUSLY_OMIT_AUTH=true to disable auth
MCP Inspector is up and running at:
http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=8eb9ddaeb36eaf6a136f1113de548348f3bdd2f900fbd34e93166124b04bd3e3
Opening browser...
```


5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

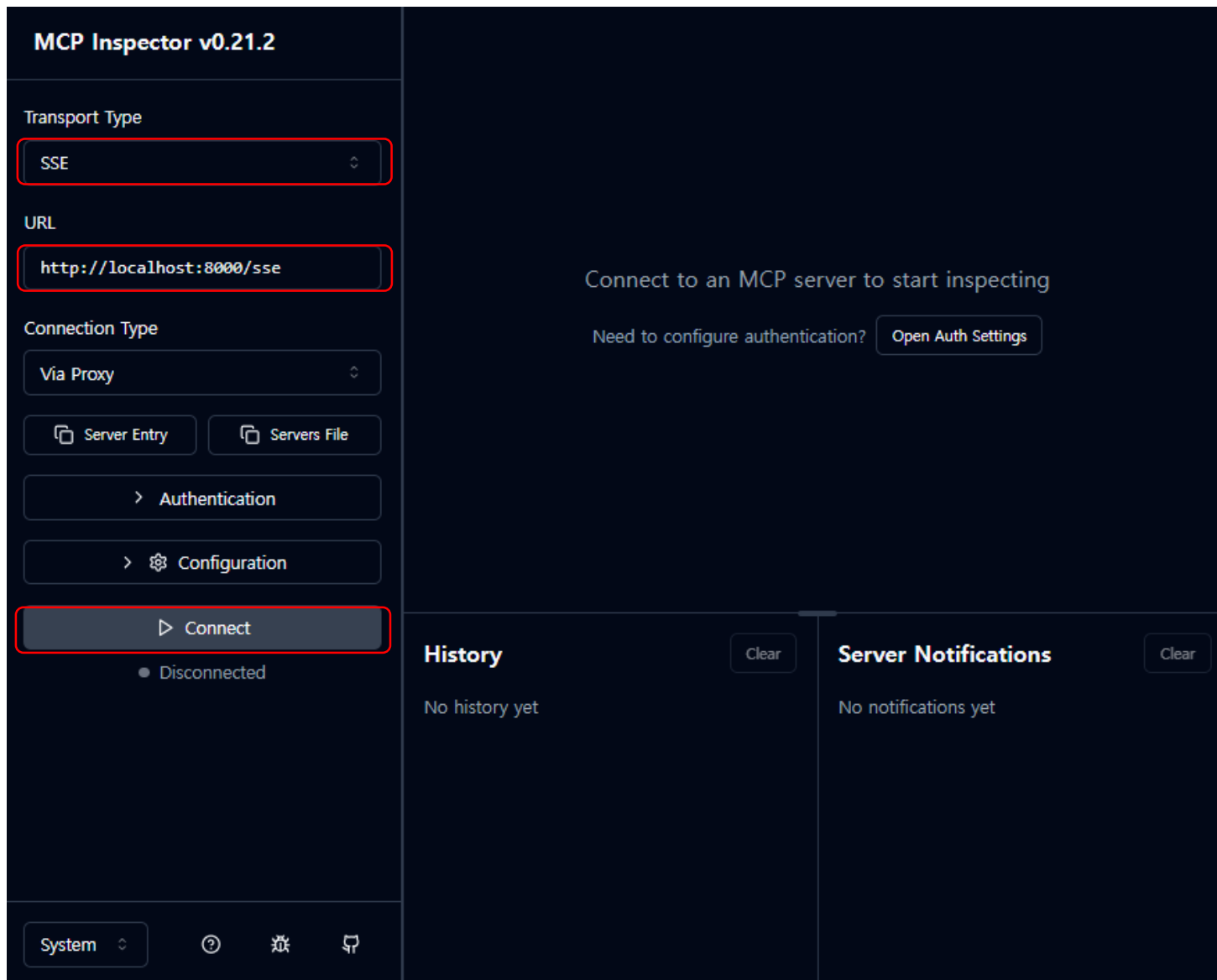
- MCP 인스펙터(inspector)



5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

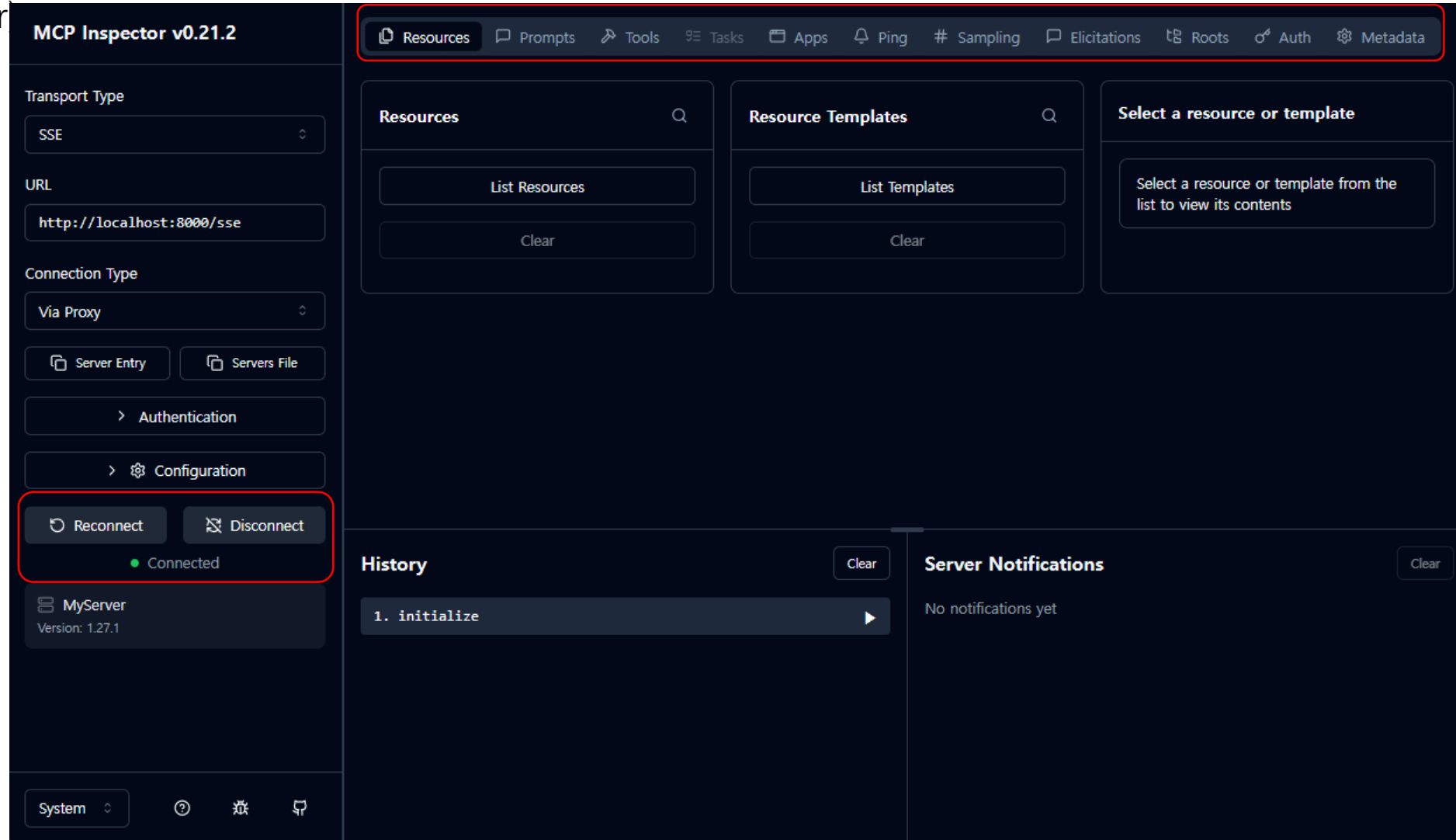
- MCP 인스펙터(inspector)



5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- MCP 인스펙터(inspector)



5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- MCP 인스펙터(inspector)

The screenshot displays the MCP Inspector v0.21.2 interface. On the left, a sidebar contains configuration options: Transport Type (SSE), URL (http://localhost:8000/sse), Connection Type (Via Proxy), and buttons for Server Entry, Servers File, Authentication, and Configuration. It also shows a Reconnect/Disconnect toggle and a 'Connected' status. The main panel is titled 'Tools' and features a search bar, a 'List Tools' button, and a 'Clear' button. Below these, the 'summarize_text' tool is listed with a description: '긴 텍스트를 요약해 간결하게 반환합니다.' and a right-pointing arrow button. The right sidebar shows the details for the 'summarize_text' tool, including its description, input fields for 'text', and a section for 'Tool-specific Metadata' with an 'Add Pair' button. Below this is the 'Output Schema' section, which displays a JSON schema for the tool's output. At the bottom, there are buttons for 'Run Tool' and 'Copy Input'. The bottom status bar shows the 'History' section with a 'Clear' button and a list of recent actions, including '2. tools/list'.

MCP Inspector v0.21.2

Transport Type: SSE

URL: http://localhost:8000/sse

Connection Type: Via Proxy

Server Entry Servers File

> Authentication

> Configuration

Reconnect Disconnect

Connected

MyServer
Version: 1.27.1

System

Tools

List Tools

Clear

summarize_text
긴 텍스트를 요약해 간결하게 반환합니다.

summarize_text

긴 텍스트를 요약해 간결하게 반환합니다.

X Read-only ✓ Destructive X Idempotent ✓ Open-world

text *

Tool-specific Metadata: Add Pair

No metadata pairs.

Output Schema: Expand

```
{
  type: "object"
  properties: {
    result: {
      type: "string"
    }
  }
}
```

Run Tool

Copy Input

History Clear

2. tools/list

Server Notifications Clear

No notifications yet

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - 요청 메시지(Request)
 - 클라이언트에서 모델이 MCP 서버에 특정 도구의 실행을 요청할 때 사용하는 메시지 형식
 - 모델은 사용자의 질의를 해석한 후, 필요한 도구의 이름과 입력값을 설정하고, 이를 기반으로 JSON-RPC 요청 메시지를 구성해서 서버에 전송

```
{
  "jsonrpc": "2.0",
  "method": "도구이름",
  "params": {
    "입력1": "값1",
    "입력2": "값2"
  },
  "id": 1
}
```

- 예를 들어, summarize_text 도구를 실행하고, text 파라미터에 특정 문장을 전달하고자 하는 경우 다음과 같이 JSON-RPC 요청 메시지를 구성할 수 있음

```
{
  "jsonrpc": "2.0",
  "method": "summarize_text",
  "params": {
    "text": "요약할 긴 문장이 여기에 들어갑니다..."
  },
  "id": 101
}
```

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - 응답 메시지(Response)
 - MCP 서버가 도구를 실행한 후, 성공 또는 실패 결과를 모델에게 전달하는 메시지 형식
 - 요청 메시지에 포함된 id 값을 그대로 반환해서 어떤 요청에 대한 응답인지 식별할 수 있도록 함

```
{
  "jsonrpc": "2.0",
  "result": "도구 실행 결과",
  "id": 1
}
```

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - 알림 메시지(Notification)
 - 요청과 유사하지만 결과를 기다리지 않는 일방향 메시지
 - 모델이 도구를 실행하되 응답을 받을 필요가 없는 경우에 사용
 - 예를 들어, 단순 로깅, 사용자 행동 기록, 내부 상태 갱신 등 비동기 처리에 적합한 상황에서 활용

```
{
  "jsonrpc": "2.0",
  "method": "log_interaction",
  "params": {
    "user": "daniel",
    "action": "searched"
  }
}
```

- 이 메시지에는 id 필드가 포함되지 않음
- 서버는 해당 메시지를 수신하면 실행만 수행하고 응답을 변환하지 않음

5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - 이처럼 JSON-RPC는 요청, 응답, 알림이라는 세 가지 메시지 유형으로 모델과 MCP 서버 간의 통신을 표준화함
 - 이 구조 덕분에 도구 호출과 관리가 간편해지고, 모델은 다양한 도구를 더욱 유연하게 활용할 수 있음

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - MCP는 기본적으로 다음 두 가지 전송 구현 방식을 제공
 1. 표준 입출력(Standard Input/Output, stdio)
 - 로컬 환경에서 CLI 기반으로 MCP 서버와 통신할 때 사용되는 방식
 - 메시지는 표준 입력으로 전달되고, 처리된 결과는 표준 출력으로 반환
 - 간단한 테스트나 내부 통합 환경에서 활용하기 좋음
 - MCP 라이브러리에서는 `mcp.run()` 호출 시 `transport` 인자를 “stdio”로 지정하면 됨
 2. 서버 전송 이벤트(Server-Sent Events, SSE)
 - HTTP 기반의 비동기 스트리밍 방식으로, 서버에서 클라이언트로 실시간 메시지를 푸시할 수 있는 기능 제공
 - 클라이언트는 HTTP POST 요청으로 메시지를 전송하고, 서버는 SSE 스트림으로 응답 메시지를 실시간으로 전달
 - 연속적인 결과나 상태 업데이트가 필요한 상황에서 안정적으로 작동하기 때문에 실제 환경에서는 기본적으로 sse 방식을 사용하는 것을 권장
 - MCP 라이브러리에서는 `mcp.run()` 호출 시 `transport` 인자를 “sse”로 지정하면 됨

5. 라마인덱스(LlamaIndex) MCP

● MCP 서버

- 메시지 형식: MCP는 모든 도구 호출 및 응답을 JSON_RPC 형식으로 주고 받음
 - MCP의 메시지 흐름 처리
 1. 모델이 요청 메시지를 생성
 - 사용자 질의에 따라 적절한 도구를 선택하고, 도구 이름과 입력값을 포함한 JSON-RPC 요청 메시지를 구성
 2. 요청 메시지가 MCP 서버로 전송됨
 - 선택된 전송 방식(stdio 또는 sse)으로 서버에 메시지가 전달
 3. 서버가 도구를 실행
 - 수신된 요청 메시지를 기반으로 해당 도구를 호출하고, 실행 결과를 생성
 4. 응답 메시지가 모델로 반환됨
 - 도구의 실행 결과를 포함한 JSON-RPC 응답 메시지가 클라이언트로 전송됨
 5. 필요 시 알람 메시지 전송
 - 결과를 기다릴 필요가 없는 작업의 경우 모델은 알림 메시지를 전송하고 서버는 이를 수신해 처리

5. 라마인덱스(LlamaIndex) MCP

- MCP 서버

- (실습) 문서 검색 에이전트 MCP

- 데이터 및 소스 코드 구조

- document_search/

- |---- server.py

- |---- client.py

- |---- llamaindex_docs/

- |---- github_docs/

5. 라마인덱스(LlamaIndex) MCP

- (실습) 문서 검색 에이전트 MCP
 - server.py 파이썬 파일을 생성하고, 라마인덱스 기반 문서 검색기 구성

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
from dotenv import load_dotenv
load_dotenv()

llama_docs = SimpleDirectoryReader("./llamaindex_docs").load_data()
llama_index = VectorStoreIndex.from_documents(llama_docs)
llama_query_engine = llama_index.as_query_engine()

github_docs = SimpleDirectoryReader("./github_docs").load_data()
github_index = VectorStoreIndex.from_documents(github_docs)
github_query_engine = github_index.as_query_engine()
```

- 라마인덱스 라이브러리를 사용해 문서를 불러오고, 벡터로 인덱싱한 뒤, 자연어 질문에 응답할 수 있는 검색 엔진을 구성
- 깃허브 관련 문서 검색기도 동일

5. 라마인덱스(LlamaIndex) MCP

- (실습) 문서 검색 에이전트 MCP

- 구현한 2개의 검색기(llama_query_engine, github_query_engine)를 어댑터를 이용해 MCP 서버에 툴로 등록

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("SearchServer")

@mcp.tool()
def search_llama_docs(query: str) -> str:
    """llamaindex 문서에서 질의에 맞는 내용을 검색합니다."""
    return llama_query_engine.query(query).response

@mcp.tool()
def search_github_docs(query: str) -> str:
    """깃허브 문서에서 질의에 맞는 내용을 검색합니다."""
    return github_query_engine.query(query).response

if __name__ == "__main__":
    mcp.run(transport="sse")
```

5. 라마인덱스(LlamaIndex) MCP

- (실습) 문서 검색 에이전트 MCP

- MCP 서버 실행

```
(.venv)  
$ python server.py
```

- 정상적으로 등록됐는지 확인하기 위해 MCP 인스펙터를 활용해 툴 목록 확인

```
$ mcp-inspector connect http://localhost:8000/sse
```

5. 라마인덱스(LlamaIndex) MCP

- (실습) 문서 검색 에이전트 MCP

- 실행 결과

The screenshot displays the MCP Inspector v0.21.2 interface, which is used for managing and interacting with Model Context Protocol (MCP) servers. The interface is divided into several panels:

- Left Panel (Configuration):** Contains settings for the transport type (SSE), URL (http://localhost:8000/sse), and connection type (Via Proxy). It also includes buttons for Server Entry, Servers File, Authentication, and Configuration. At the bottom, there are Reconnect and Disconnect buttons, and a status indicator showing "Connected".
- Top Panel (Tools):** Lists available tools. The "search_llama_docs" tool is highlighted with a red box. It is described as "llamaindex 문서에서 질의에 맞는 내용을 검색합니다." (Searches for content matching the query in the llamaindex document).
- Right Panel (Tool Details):** Provides details for the selected "search_llama_docs" tool. It includes a query input field, a "Tool-specific Metadata" section (currently empty), and an "Output Schema" section showing a JSON object type.
- Bottom Panel (History):** Shows a list of recent actions, including "2. tools/list" and "1. initialize".

5. 라마인덱스(LlamaIndex) MCP

● MCP 클라이언트

- AI 어플리케이션 내부에 존재하면서 사용자의 요청이나 모델의 내부 판단에 따라 외부 MCP 서버에 명령을 전달하는 역할
- 여기서 명령을 전달한다는 것은 단순히 데이터를 전송하는 것이 아니라 특정 도구(function)를 이름과 입력값을 기반으로 호출하는 평션 콜링을 수행한다는 의미
- 예를 들어, 사용자가 ‘라마인덱스를 활용하는 방법을 알려주세요’라고 요청하면 AI 모델은 이 요청이 외부 문서 검색 도구로 해결 가능하다고 판단할 수 있음
- 이때 MCP 클라이언트는 라마인덱스 문서 검색 서버(MCP 서버)에 평션 콜링을 보내고, 검색 결과를 다시 사용자에게 자연스럽게 전달
- 따라서 MCP 클라이언트는 사용자의 자연어 요청을 해석한 모델의 의사결정과 실제 외부 도구(MCP 서버) 호출 과정을 연결하는 중간 다리 역할을 함
- 라마인덱스 MCP에서는 이러한 MCP 클라이언트를 간단히 활용할 수 있도록 BasicMCPClient라는 클래스를 제공
- BasicMCPClient는 복잡한 HTTP 통신, API 요청 구성, 에러 처리등을 내부적으로 모두 관리해 주기 때문에 사용자는 단순히 메서드를 호출하는 것만으로 서버에 등록된 도구를 사용할 수 있음

5. 라마인덱스(LlamaIndex) MCP

- MCP 클라이언트

- list_tools.py 실행

(.venv)

\$ python list_tools.py

```
{
  "name": "search_llama_docs",
  "description": "llamaindex 문서에서 질의에 맞는 내용을 검색합니다.",
  "parameters": {
    "properties": {
      "query": {
        "description": "",
        "title": "Query",
        "type": "string"
      }
    }
  },
  "required": [
    "query"
  ],
  "type": "object"
}
```

```
{
  "name": "search_github_docs",
  "description": "깃허브 문서에서 질의에 맞는 내용을 검색합니다.",
  "parameters": {
    "properties": {
      "query": {
        "description": "",
        "title": "Query",
        "type": "string"
      }
    }
  },
  "required": [
    "query"
  ],
  "type": "object"
}
```

5. 라마인덱스(LlamaIndex) MCP

- MCP 클라이언트

- list_tools.py 실행
 - 라마인덱스는 MCP 서버에 등록된 도구들을 자동으로 가져오고, 이를 에이전트가 이해할 수 있는 형태로 변환하는 기능 제공
 - 이때 핵심이 되는 클래스가 McpToolSpec
 - McpToolSpec은 내부적으로 MCPClient를 통해 MCP 서버의 /list_tools 엔드포인트를 호출해 도구 목록을 가져오고, 각 도구를 라마인덱스에서 사용하는 FunctionTool 객체로 변환
 - FunctionTool은 파이썬 함수나 외부 도구를 감싸면서 도구명, 설명, 입력 스키마 등을 자동으로 구성해 라마인덱스가 자연스럽게 호출할 수 있도록 하는 역할을 함
 - 이 과정을 코드로 살펴봄

5. 라마인덱스(LlamaIndex) MCP

- MCP 클라이언트

- client.py

```
from llama_index.tools.mcp import BasicMCPClient
from dotenv import load_dotenv
load_dotenv()

# MCP 서버 연결
mcp_client = BasicMCPClient("http://localhost:8000/sse")

from llama_index.tools.mcp import McpToolSpec

# MCP 도구를 LlamaIndex용으로 변환
tool_spec = McpToolSpec(client=mcp_client)

# FunctionTool 리스트 획득
tools = tool_spec.to_tool_list()
```

- <http://localhost:8000>은 server.py에서 구현한 MCP 서버를 의미
- 라마인덱스에서 MCP 서버의 도구들을 사용할 수 있도록 McpToolSpec 객체 생성
- tool_spec 객체는 내부적으로 mcp_client를 사용해 MCP 서버의 /list_tools API를 호출해 등록된 도구 목록을 가져오고, 이를 라마인덱스가 이해할 수 있는 FunctionTool 객체로 변환하는 역할을 함
- to_list_tools() 메서드를 호출하면 변환된 FunctionTool 객체들의 리스트가 tools 변수에 저장됨
- 이후 이 tools 리스트를 라마인덱스 에이전트에 등록하면 MCP 서버의 외부 도구를 내부 함수처럼 자연스럽게 사용할 수 있음

5. 라마인덱스(LlamaIndex) MCP

- MCP 클라이언트

- 에이전트 구성(client.py)

```
from llama_index.llms.openai import OpenAI
from llama_index.core.agent.workflow import FunctionAgent

# LLM 설정 (gpt-4o-mini 사용)
llm = OpenAI(model="gpt-4o-mini")

# FunctionAgent 생성
agent = FunctionAgent(
    tools=tools,
    llm=llm,
    system_prompt=(
        "사용자의 질문을 분석하여, LlamaIndex 문서나 GitHub 문서 중 적절한 문서를 검색하는 도구를 선택하고, "
        "그 결과를 바탕으로 구체적인 예시와 함께 답변을 생성하는 에이전트입니다."
    ),
)
```

- OpenAI 클래스는 라마인덱스에서 OpenAI 기반 모델을 사용하기 위한 래퍼 클래스
- FunctionAgent는 다양한 도구와 LLM을 결합해 자연어 질문을 분석하고 적절한 도구를 선택한 뒤, 그 결과를 기반으로 답변을 생성하는 역할 수행
- tools=tools는 MCP 서버에서 가져온 도구 리스트를 에이전트에 연결하고, llm=llm은 앞서 설정한 OpenAI 모델을 연결함

5. 라마인덱스(LlamaIndex) MCP

● MCP 클라이언트

- 에이전트를 이용해 실제로 질문을 던지고, 어떤 도구를 선택해 어떻게 답변을 생성하는 확인(client.py)

```
import asyncio

async def run_agent():
    question = "Llama-Index를 활용하여 문서 검색하는 방법"
    response = await agent.run(question)
    print("\n=== 에이전트 응답 ===\n")
    print(response)

if __name__ == "__main__":
    asyncio.run(run_agent())
```

- OpenAI 클래스는 라마인덱스에서 OpenAI 기반 모델을 사용하기 위한 래퍼 클래스
- FunctionAgent는 다양한 도구와 LLM을 결합해 자연어 질문을 분석하고 적절한 도구를 선택한 뒤, 그 결과를 기반으로 답변을 생성하는 역할 수행
- tools=tools는 MCP 서버에서 가져온 도구 리스트를 에이전트에 연결하고, llm=llm은 앞서 설정한 OpenAI 모델을 연결함

5. 라마인덱스(LlamaIndex) MCP

● MCP 클라이언트

- 에이전트를 이용해 실제로 질문을 던지고, 어떤 도구를 선택해 어떻게 답변을 생성하는 확인(client.py)

```
import asyncio

async def run_agent():
    question = "Llama-Index를 활용하여 문서 검색하는 방법"
    response = await agent.run(question)
    print("\n=== 에이전트 응답 ===\n")
    print(response)

if __name__ == "__main__":
    asyncio.run(run_agent())
```

- OpenAI 클래스는 라마인덱스에서 OpenAI 기반 모델을 사용하기 위한 래퍼 클래스
- FunctionAgent는 다양한 도구와 LLM을 결합해 자연어 질문을 분석하고 적절한 도구를 선택한 뒤, 그 결과를 기반으로 답변을 생성하는 역할 수행
- tools=tools는 MCP 서버에서 가져온 도구 리스트를 에이전트에 연결하고, llm=llm은 앞서 설정한 OpenAI 모델을 연결함

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- MCP를 활용해 사용자 질문에 따라 실시간 오늘과 내일의 날씨를 안내하는 날씨 에이전트를 만들어 봄
- 단순 기능 구현에 그치지 않고, MCP 서버를 구축하고 외부 날씨 API를 MCP 도구로 등록한 뒤, 클라이언트가 LLM으로 자연어를 분석해서 필요한 도구를 선택하고 호출하는 전체 구조를 구성하도록 함

- 폴더 구성

```
weather/
```

```
|---- weather-server.py
```

```
|---- weather-client.py
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- OpenWeatherMap API 키 발급받기

- OpenWeatherMap은 전 세계 기상 데이터를 제공하는 서비스로, 회원가입만 하면 무료로 API 키를 발급 받을 수 있음

- <https://openweathermap.org/>



6. 날씨 에이전트 구현

● (실습) 날씨 에이전트

- 현재 날씨 조회를 위한 API 확인



Weather in your city

[Guide](#)[API](#)[Dashboard](#)[Marketplace](#)[Pricing](#)[Maps](#)[Our Initiatives](#)[Partners](#)[Blog](#)[For Business](#)

Current & Forecast

Current Weather Data

- Access current weather data for any location
- We collect and process weather data from different sources such as global and local weather models, satellites, radars and a vast network of weather stations
- JSON, XML, and HTML formats
- Included in both free and paid subscriptions

[Subscribe](#)[API Docs](#)

Current & Forecast

Hourly Forecast 4 days

- Hourly forecast is available for 4 days
- Forecast weather data for 96 timestamps
- JSON and XML formats
- Included in the Developer, Professional and Expert subscription plans

[Subscribe](#)[API Docs](#)

Current & Forecast

Daily Forecast 16 days

- 16 days forecast is available for any location on the globe
- 1-day step for 16 days
- JSON and XML formats
- Included in all paid subscription plans

[Subscribe](#)[API Docs](#)

Current & Forecast

Climatic Forecast 30 days

- Forecast weather data for 30 days
- JSON format
- Included in the Developer, Professional and Expert subscription plans

Current & Forecast

Bulk Current Data

- Current weather, a variety of weather forecasts and their 7-day archive via regularly updated files
- Weather bulks are grouped by types of weather data and location lists (global city lists or ZIP code lists of EU, UK, and

Current & Forecast

5 Day / 3 Hour Forecast

- 5 day forecast for any location on the globe
- 5 day forecast with a 3-hour step
- JSON and XML formats
- Included in both free and paid subscriptions

Call current weather data

How to make an API call

```
https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key}
```

Parameters

lat	required	Latitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
lon	required	Longitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
appid	required	Your unique API key (you can always find it on your account page under the "API key" tab)
mode	optional	Response format. Possible values are <code>xml</code> and <code>html</code> . If you don't use the <code>mode</code> parameter format is JSON by default. Learn more

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)

```
from dotenv import load_dotenv
from llama_index.llms.openai import OpenAI
from mcp.server.fastmcp import FastMCP
from llama_index.core.llms import ChatMessage
import requests
import os

load_dotenv()

weatherapi_key = os.getenv("WEATHER_API_KEY")
llm = OpenAI(model="gpt-4o-mini")

mcp = FastMCP('weather-server')
```

- .env 설정 파일에 WEATHER_API_KEY 변수 이름으로 key 값 저장하고, 불러옴
- FastMCP 인스턴스는 'weather-server'라는 이름으로 설정했음

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)
 - 도시명 추출하기

```
def extract_city_name_with_llm(query: str) -> str:
    prompt = (
        f"다음 문장에서 날씨를 묻는 도시명이 있다면 그 도시명을 영어로만 출력하세요. "
        f"도시가 없다면 반환하지 않습니다. 오직 도시명 하나만 영어로 출력하세요.\n"
        f"예: Seoul, Busan, Jeju\n"
    )

    messages = [
        ChatMessage(role="system", content=prompt),
        ChatMessage(role="user", content=query),
    ]

    response = llm.chat(
        messages=messages,
        model="gpt-4o-mini",
        temperature=0,
        max_tokens=10,
    )
    return response.message.content.strip()
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- MCP 서버 구축(weather-server.py)

- OpenWeatherMap API 연동

```
def get_geo(city: str, limit: int=1) -> str:
    """도시명을 입력받아 OpenWeatherMap 지오코딩 API를 호출하여 위도와 경도를 가져옴"""
    url = f"http://api.openweathermap.org/geo/1.0/direct?q={city}&limit={limit}&appid={weatherapi_key}"

    res = requests.get(url)
    data = res.json()
    return data[0]
```

- q(필수) : 조회하고자 하는 도시명
- limit(선택) : 결과의 최대 개수를 지정. 예를 들어, 동일한 이름을 가진 여러 도시 중 최대 5개까지 반환하도록 설정할 수 있음
- appid(필수) : 발급받은 OpenWeatherMap API key

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)
 - 오늘의 날씨 조회

```
def get_today_weather(city: str) -> str:
    """[오늘 날씨 조회 API] OpenWeatherMap current weather API를 이용해 오늘 현재 날씨를 조회"""
    geo = get_geo(city)
    url = f"https://api.openweathermap.org/data/2.5/weather?lat={geo['lat']}&lon={geo['lon']}&appid={weatherapi_key}"

    res = requests.get(url)
    if res.status_code != 200:
        return f"{city}의 날씨 정보를 가져오지 못했습니다."

    data = res.json()

    city = data["name"]
    temp_k = data["main"]["temp"]
    feelslike_k = data["main"]["feels_like"]
    condition = data["weather"][0]["description"]

    # 켈빈(K) -> 섭씨(°C)로 변환
    temp_c = round(temp_k - 273.15, 1)
    feelslike_c = round(feelslike_k - 273.15, 1)

    return f"{city}의 현재 날씨는 {condition}, 기온은 {temp_c}°C, 체감온도는 {feelslike_c}°C입니다."
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)
 - 내일의 날씨 조회

Call hourly forecast data

How to make an API call

You can search weather forecast for 4 days with data every hour by geographic coordinates.

All weather data can be obtained in JSON and XML formats.

API call

```
https://pro.openweathermap.org/data/2.5/forecast/hourly?lat={lat}&lon={lon}&appid={API key}
```



Parameters

lat	required	Latitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
lon	required	Longitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
appid	required	Your unique API key (you can always find it on your account page under the "API key" tab)

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- MCP 서버 구축(weather-server.py)

- 내일의 날씨 조회

```
from datetime import datetime, timedelta

def get_tomorrow_weather(city: str) -> str:
    """[내일 날씨 조회 API] OpenWeatherMap forecast API를 이용해 내일 날씨를 조회"""
    geo = get_geo(city)
    url = f"https://api.openweathermap.org/data/2.5/forecast?lat={geo['lat']}&lon={geo['lon']}&appid={weatherapi_key}"

    res = requests.get(url)
    if res.status_code != 200:
        return f"{city}의 내일 날씨 정보를 가져오지 못했습니다."

    data = res.json()
    tomorrow = datetime.utcnow() + timedelta(days=1)
    tomorrow_str = tomorrow.strftime("%Y-%m-%d")

    # 내일 중 정오 (12:00:00) 시간 데이터 우선
    tomorrow_forecasts = [item for item in data["list"] if item["dt_txt"].startswith(tomorrow_str)]
    target = next((item for item in tomorrow_forecasts if "12:00:00" in item["dt_txt"]), tomorrow_forecasts[0])

    temp_k = target["main"]["temp"]
    condition = target["weather"][0]["description"]
    temp_c = round(temp_k - 273.15, 1)

    return f"{city}의 내일 날씨는 {condition}, 예상 기온은 {temp_c}°C입니다."
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)
 - MCP 도구 등록

```
@mcp.tool()
def today_weather_query(city: str = None) -> str:
    """[MCP Tool] 파라미터로 받은 도시 또는 세션의 도시로 오늘 날씨를 반환"""
    print(f"[today_weather_query] city={city}")

    if city is None:
        return "도시 정보가 없습니다. 먼저 도시명을 알려주세요."

    return get_today_weather(city)

@mcp.tool()
def tomorrow_weather_query(city: str = None) -> str:
    """[MCP Tool] 파라미터로 받은 도시 또는 세션의 도시로 내일 날씨를 반환"""
    print(f"[tomorrow_weather_query] city={city}")

    if city is None:
        return "도시 정보가 없습니다. 먼저 도시명을 알려주세요."

    return get_tomorrow_weather(city)
```


6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - MCP 서버 구축(weather-server.py)
 - MCP 서버 실행

```
if __name__ == "__main__":  
  
    query = "서울 오늘 날씨 알려줘"  
    print(extract_city_name_with_llm(query))  
  
    query = "부산 내일 비오나요?"  
    print(extract_city_name_with_llm(query))  
  
    city_info = get_geo("Seoul")  
    print(city_info['lat'], city_info['lon'])  
  
    today_weather = get_today_weather("Seoul")  
    print(today_weather)  
  
    tomorrow_weather = get_tomorrow_weather("Seoul")  
    print(tomorrow_weather)  
  
    mcp.run(transport="sse")
```

(.venv)

\$ python weather-server.py

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- MCP 클라이언트 구현하기: 날씨 질문하기(weather-client.py)

```
from dotenv import load_dotenv
load_dotenv()

from llama_index.tools.mcp import BasicMCPClient, McpToolSpec
from llama_index.llms.openai import OpenAI
from llama_index.core.agent.workflow import FunctionAgent
import asyncio

mcp_client = BasicMCPClient("http://localhost:8000/sse")
tool_spec = McpToolSpec(client=mcp_client)
tools = tool_spec.to_tool_list()
```

- 서버 구성이 완료됐으므로 이제 클라이언트 측에서는 사용자의 질문을 받아 MCP 서버에 등록된 도구를 통해 오늘과 내일의 날씨를 조회하는 흐름을 구성
- 라마인덱스 MCP에서 클라이언트는 BasicMCPClient를 통해 MCP 서버와 연결하고, 서버에 등록된 도구 목록을 자동으로 가져올 수 있음
- 가져온 도구들은 LLM과 연동된 FunctionAgent에 등록되어 사용자 질문에 따라 적절한 도구를 자동으로 선택하고 호출하는 방식으로 작동

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트
 - LLM과 시스템 프롬프트 설정(weather-client.py)

```
llm = OpenAI(model="gpt-4o-mini")

prompt = """당신은 날씨 정보를 알려주는 에이전트입니다.
사용자가 질문한 도시와 시간(오늘/내일)에 따라 적절한 날씨 도구를 호출해야 합니다.
도시명을 사용할 때는 영어로 변환된 도시명을 사용하며,
사용자에게 반환하는 답변은 항상 한국어로 자연스럽게 작성합니다.
"""

agent = FunctionAgent(
    tools=tools,
    llm=llm,
    system_prompt=prompt,
)
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- 실제 사용자 질문을 입력받고 에이전트가 응답을 생성하는 비동기 함수 작성 및 실행(weather-client.py)

```
async def run_agent():
    questions = [
        "오늘 날씨 어때?",
        "서울이에요",
        "내일 부산 날씨는?",
    ]

    for q in questions:
        print(f"사용자: {q}")
        result = await agent.run(q)
        print(f"에이전트: {result}")

if __name__ == "__main__":
    asyncio.run(run_agent())
```

(.venv)

\$ python weather-client.py

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- 실행 결과

사용자: 오늘 날씨 어때?

에이전트: 어떤 도시의 날씨를 알고 싶으신가요? 도시명을 말씀해 주시면 그에 맞는 날씨 정보를 제공해 드리겠습니다.

사용자: 서울이에요

에이전트: 오늘과 내일 중 어떤 날씨 정보를 원하시나요?

사용자: 내일 부산 날씨는?

에이전트: 내일 부산의 날씨는 맑고, 예상 기온은 19.0°C입니다.

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- 실행 결과(MCP 서버 로그)

```
INFO: Started server process [31712]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: 127.0.0.1:62923 - "GET /sse HTTP/1.1" 200 OK
INFO: 127.0.0.1:62925 - "POST /messages/?session_id=c1cbb70d80994b00a5d82054cc5388ea HTTP/1.1" 202 Accepted
INFO: 127.0.0.1:62925 - "POST /messages/?session_id=c1cbb70d80994b00a5d82054cc5388ea HTTP/1.1" 202 Accepted
INFO: 127.0.0.1:62925 - "POST /messages/?session_id=c1cbb70d80994b00a5d82054cc5388ea HTTP/1.1" 202 Accepted
[05/16/26 02:12:02] INFO Processing request of type ListToolsRequest server.py:727
INFO: 127.0.0.1:60203 - "GET /sse HTTP/1.1" 200 OK
INFO: 127.0.0.1:60205 - "POST /messages/?session_id=91f885b0464f4e3ab70d379f05881166 HTTP/1.1" 202 Accepted
INFO: 127.0.0.1:60205 - "POST /messages/?session_id=91f885b0464f4e3ab70d379f05881166 HTTP/1.1" 202 Accepted
INFO: 127.0.0.1:60205 - "POST /messages/?session_id=91f885b0464f4e3ab70d379f05881166 HTTP/1.1" 202 Accepted
[05/16/26 02:12:06] INFO Processing request of type CallToolRequest server.py:727
[tomorrow_weather_query] city=Busan
[05/16/26 02:12:07] INFO Warning: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for server.py:717
removal in a future version. Use timezone-aware objects to represent datetimes in UTC:
datetime.datetime.now(datetime.UTC).
INFO: 127.0.0.1:60205 - "POST /messages/?session_id=91f885b0464f4e3ab70d379f05881166 HTTP/1.1" 202 Accepted
INFO Processing request of type ListToolsRequest server.py:727
```

6. 날씨 에이전트 구현

- (실습) 날씨 에이전트

- 클라이언트가 실제로 서버에 요청을 보낼 때마다 서버는 다음과 같은 형태의 로그를 출력
 - ListToolsRequest
 - 클라이언트가 MCP 서버에 등록된 도구 목록을 요청할 때 발생
 - 서버는 도구 리스트를 반환해서 클라이언트가 사용할 수 있는 도구를 인식할 수 있도록 함
 - CallToolRequest
 - 클라이언트가 특정 도구를 호출할 때 발생
 - MCP 서버는 해당 도구(today_weather_query 또는 tomorrow_weather_query)를 실행하고, 결과를 반환
- 이처럼 MCP는 내부적으로 'ListToolsRequest -> CallToolRequest -> 도구 호출 -> 응답 반환' 이라는 순서로 요청을 처리함